

QA Automation - Coding Challenge

Estimated time: 4–6 hours.

Task 1 - Web UI Automation Framework

Target Application

<https://trade.mb.io/>

This is a public-facing platform. All required scenarios are completable without logging in. Do not create accounts or submit any personal or financial information.

Core Test Scenarios

Build an automation framework that validates the core functionality and demonstrates good testing practice. You may use any modern framework (e.g. Playwright, Cypress, Selenium) but your design should be scalable and easy to extend.

1. Navigation & Layout

- Top navigation renders with all expected items visible
- Each navigation item links to the correct destination
- Navigation behaves correctly at standard desktop viewport sizes

2. Trading Functionality

- Spot trading section renders and displays trading pairs
- Trading pairs are correctly grouped into categories
- Trading pair entries contain the expected data fields

3. Content & Links

- Marketing banners render in the expected page region
- App Store and Google Play download links resolve correctly
- About Us > Why MultiBank page renders all expected components with correct headings and section text

4. Negative / Edge Cases

Include at least two of the following:

- Invalid route handling
- Broken link detection across navigation links
- Viewport regression at a mobile breakpoint
- Content loading timeout handling

Bonus (Optional):

- Validate relevant API/network responses
- Add visual regression checks
- Parameterize test data
- Integrate basic CI (e.g. GitHub Actions)

Evaluation Criteria

Area	What we assess
Architecture	Maintainability and separation of concerns
Test design	Independence, determinism, and scope
Assertions	Meaningful and resilient
Edge Cases	Awareness of real-world failure modes
Code quality	Readability, consistency, professional finish

Task 2 - QA Strategy & Thinking

The Scenario

You've just joined a fintech startup as a QA Engineer. On day one you're handed a mobile trading app, for both iOS and Android, that's two weeks from its first public release. There's no existing test suite, no QA documentation, and the dev team has been shipping fast. Assume real user funds are involved.

Answer the following in your own words:

1. Where do you start?
2. How would you approach testing this app?
3. What does QA look like inside a sprint, from ticket creation through to regression?
4. What does your ideal regression suite look like?

5. What would keep you up at night about this app specifically and releasing to the public?

Deliverables

- GitHub repository with clear commit history and a README that lets a reviewer run your suite with a single command
- Sample test execution report and evidence of cross-browser runs
- Written responses to Task 2 (included in the repo README or as a separate doc)
- Include framework/design decisions and any assumptions made
- Provide:
 - A test plan
 - A release readiness checklist
 - A risk matrix

Submission

Share your GitHub repository link. Your README is the first thing we read.